

NIT2112

Object Oriented Programming

Session 09

AI-Assisted Refactoring and Testing

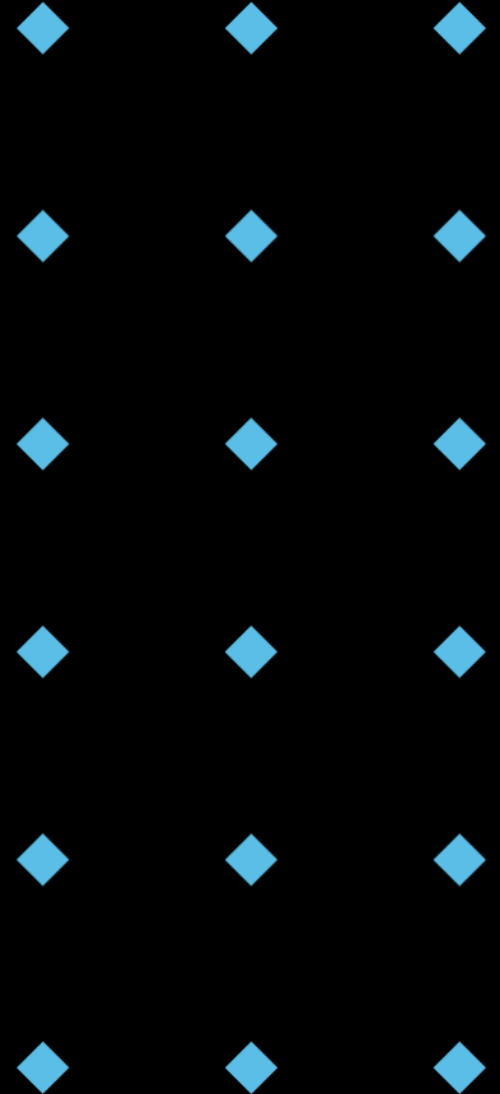
Acknowledgement of Country



Victoria University acknowledges, recognises and respects the Ancestors, Elders and families of the Bunurong/Boonwurrung, Wadawurrung and Wurundjeri/Woiwurrung of the Kulin who are the traditional owners of University land in Victoria, the Gadigal and Guring-gai of the Eora Nation who are the traditional owners of University land in Sydney, and the Yugara/YUgarapul people and Turrbal people living in Meanjin (Brisbane).

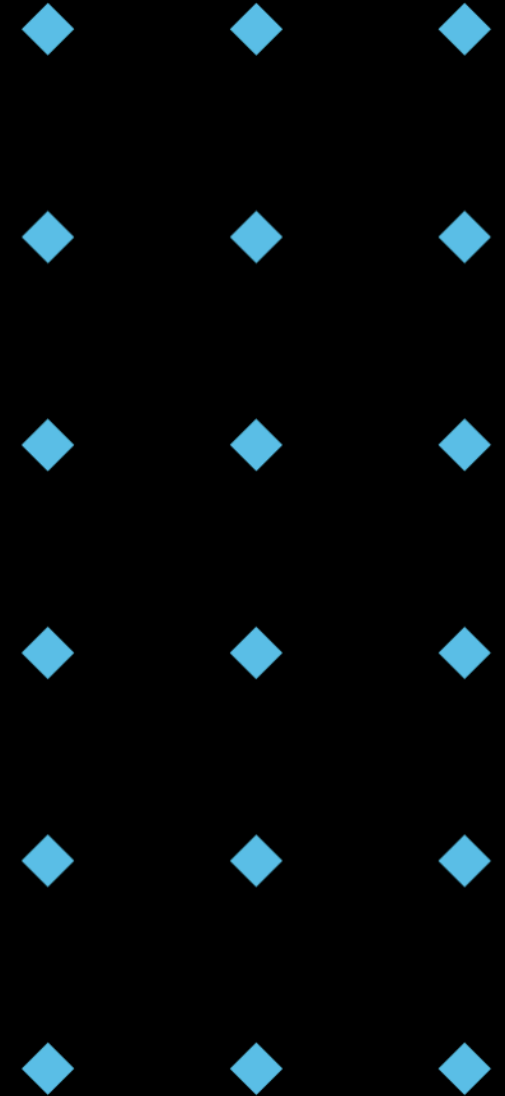
Session Outline

- ◆ AI-Assisted Refactoring
- ◆ AI for Unit Test Generation
- ◆ AI Support for TDD & Mocking
- ◆ Using AI for Edge Case Identification
- ◆ AI Insights for Test Coverage & Integration



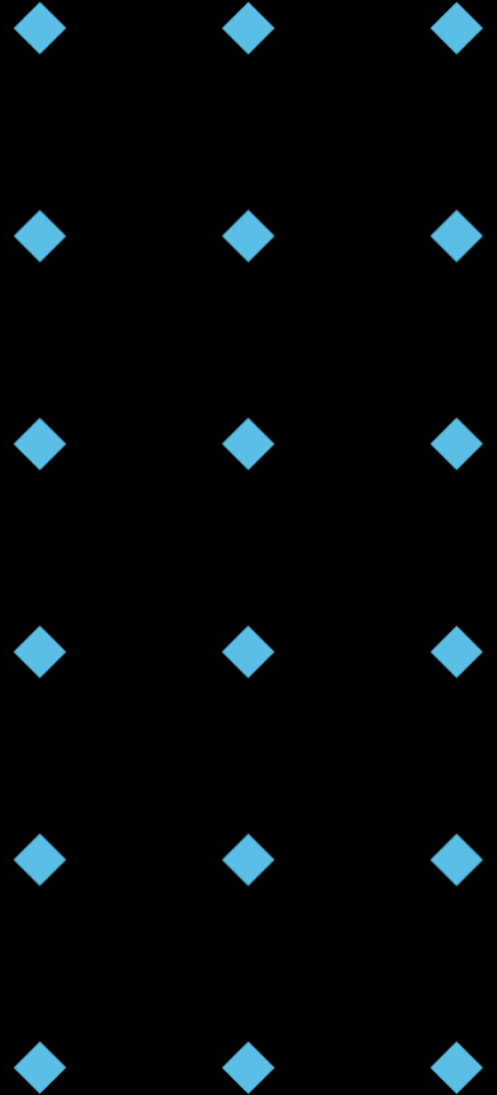
The Importance of Testing & Refactoring in OOP

- ◆ Refactoring:
 - ◆ Improves code readability and maintainability.
 - ◆ Reduces complexity.
 - ◆ Makes code easier to extend and modify.
 - ◆ Can sometimes improve performance (but clarity is often the primary goal).
 - ◆ Crucial: Refactoring should **not** change observable behaviour (tests are key!).
- ◆ Testing:
 - ◆ Ensures code correctness (does what it's supposed to do).
 - ◆ Prevents regressions (breaking existing functionality).
 - ◆ Facilitates fearless refactoring and adding new features.
 - ◆ Improves design (writing testable code often leads to better structure).
 - ◆ Acts as living documentation.



Refactoring Fundamentals

- ◆ Definition: **Restructuring** existing computer code – changing the factoring – **without** changing its external behaviour.
- ◆ Goals:
 - ◆ Improve **nonfunctional** attributes: **readability, complexity, maintainability, extensibility**.
- ◆ Common Techniques:
 - ◆ Extract Method/Class
 - ◆ Rename Variable/Method/Class
 - ◆ Introduce Parameter Object
 - ◆ Replace Conditional with Polymorphism
 - ◆ Simplify Conditional Logic
- ◆ Pre-requisite: Good test suite!



AI's Role in Refactoring

- ◆ **Suggestion Generation:**

- ◆ Suggesting refactoring opportunities based on code analysis (e.g., identifying long methods, complex conditionals).
- ◆ Offering specific refactoring implementations (e.g., "Extract this block to a new method named *'calculate_discount'*").

- ◆ **Code Transformation:**

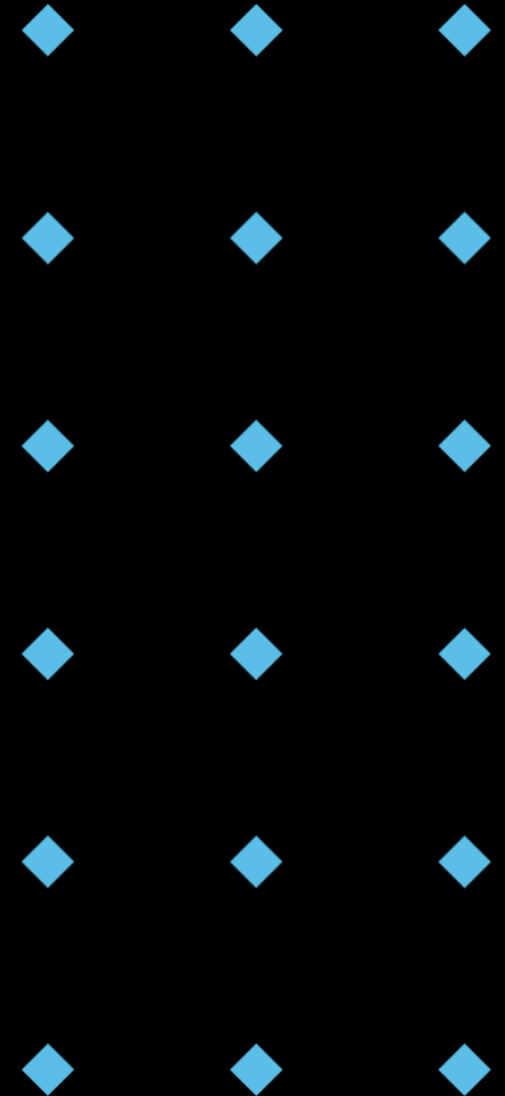
- ◆ Applying standard refactorings automatically or semi-automatically based on prompts or suggestions.

- ◆ **Explanation:**

- ◆ Some tools might explain why a refactoring is suggested (e.g., "This improves readability").

- ◆ **Performance Hints:**

- ◆ AI might suggest alternative implementations that could be more performant, but caution is needed. Verify performance claims.



Example: Refactoring with AI's Help

Original Code (Long Method)

```
1 class ReportGenerator:
2     def generate(self, data):
3         # Section 1: Data validation
4         print("Validating data...")
5         if not data:
6             raise ValueError("No data provided")
7         for item in data:
8             if item < 0:
9                 print(f"Warning: Negative value found {item}")
10        print("Data validation complete.")
11
12        # Section 2: Data processing
13        print("Processing data...")
14        processed_data = [item * 2 for item in data if item >= 0]
15        print("Data processing complete.")
16
17        # Section 3: Report formatting
18        print("Formatting report...")
19        report = "Report:\n" + "\n".join(map(str, processed_data))
20        print("Report formatting complete.")
21        return report
```

[Provide Code]
Refactor: Extract Method

Potential AI Output:



```
1 class ReportGenerator:
2     def _validate_data(self, data):
3         print("Validating data...")
4         if not data:
5             raise ValueError("No data provided")
6         for item in data:
7             if item < 0:
8                 print(f"Warning: Negative value found {item}")
9         print("Data validation complete.")
10
11     def _process_data(self, data):
12         print("Processing data...")
13         processed_data = [item * 2 for item in data if item >= 0]
14         print("Data processing complete.")
15         return processed_data
16
17     def _format_report(self, processed_data):
18         print("Formatting report...")
19         report = "Report:\n" + "\n".join(map(str, processed_data))
20         print("Report formatting complete.")
21         return report
22
23     def generate(self, data):
24         self._validate_data(data)
25         processed_data = self._process_data(data)
26         report = self._format_report(processed_data)
27         return report
```

Example: AI Simplifying Conditionals

Original Code

```
1 def get_user_status(user_points, is_premium, last_login_days):
2     if user_points > 1000:
3         if is_premium:
4             if last_login_days < 7:
5                 return "Gold Tier - Active Premium"
6             else:
7                 return "Gold Tier - Inactive Premium"
8         else:
9             return "Gold Tier - Standard"
10    elif user_points > 500:
11        if is_premium:
12            return "Silver Tier - Premium"
13        else:
14            # Nested check
15            if last_login_days < 30:
16                return "Silver Tier - Active Standard"
17            else:
18                return "Silver Tier - Inactive Standard"
19    else:
20        return "Bronze Tier"
```

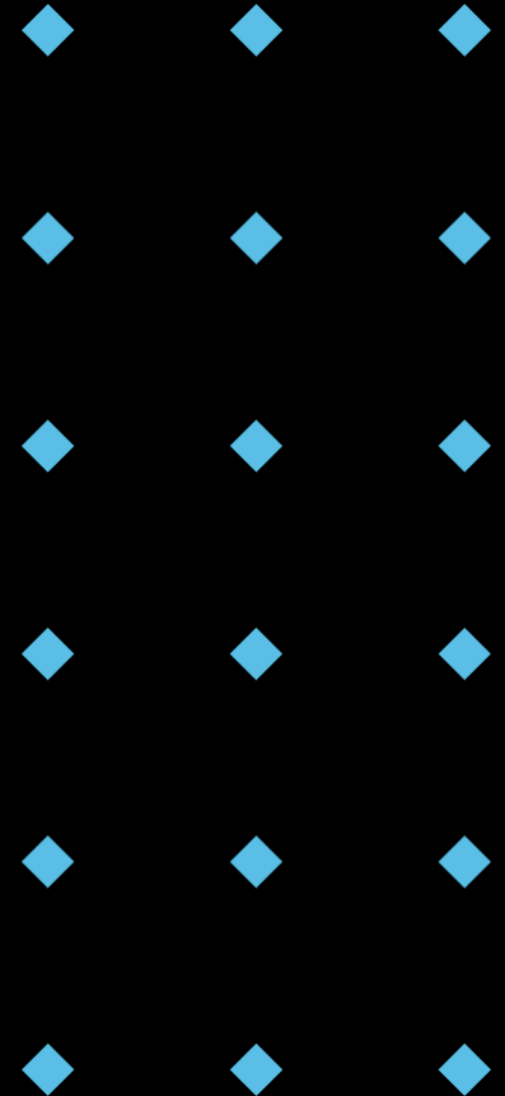
[Provide Code]
Refactor: Simplify conditional logic

Potential AI Output:

```
1 # This refactoring:
2 # 1. Extracting the complex status logic to a separate helper function
3 # 2. Making the main function more focused on just determining the tier
4 # 3. Creating a clearer separation of responsibilities between functions
5 # 4. Reducing nesting depth, making the code easier to read and maintain
6 def get_user_status(user_points, is_premium, last_login_days):
7     # Determine tier based on points
8     if user_points > 1000:
9         tier = "Gold"
10    elif user_points > 500:
11        tier = "Silver"
12    else:
13        return "Bronze Tier" # Bronze tier has no subtypes
14
15    # For non-Bronze tiers, determine full status
16    return f"{tier} Tier - {get_status_suffix(tier, is_premium, last_login_days)}"
17
18 def get_status_suffix(tier, is_premium, last_login_days):
19     if is_premium:
20         if tier == "Gold":
21             return "Active Premium" if last_login_days < 7 else "Inactive Premium"
22         return "Premium" # Silver Premium has no activity indicator
23     else:
24         if tier == "Gold":
25             return "Standard" # Gold Standard has no activity indicator
26         # Silver Standard
27         return "Active Standard" if last_login_days < 30 else "Inactive Standard"
```

AI and Performance Refactoring

- ◆ Potential AI Assistance:
 - ◆ Identifying **inefficient loops** or **data structures**.
 - ◆ Suggesting **alternative algorithms** (e.g., using a dictionary for lookups instead of list iteration).
 - ◆ Pointing out potentially **expensive operations** inside loops.
- ◆ Important Notes:
 - ◆ **Measure First:** Optimize with profiling. Use tools like cProfile.
 - ◆ **AI Doesn't Know Your Context:** An AI suggestion might be faster in general but slower for your specific data patterns or hardware.
 - ◆ **Readability vs. Performance:** Don't sacrifice clear, maintainable code for marginal gains unless performance is a proven bottleneck.
 - ◆ **Verify AI Suggestions:** Always understand why the suggestion is supposedly faster and test it rigorously.

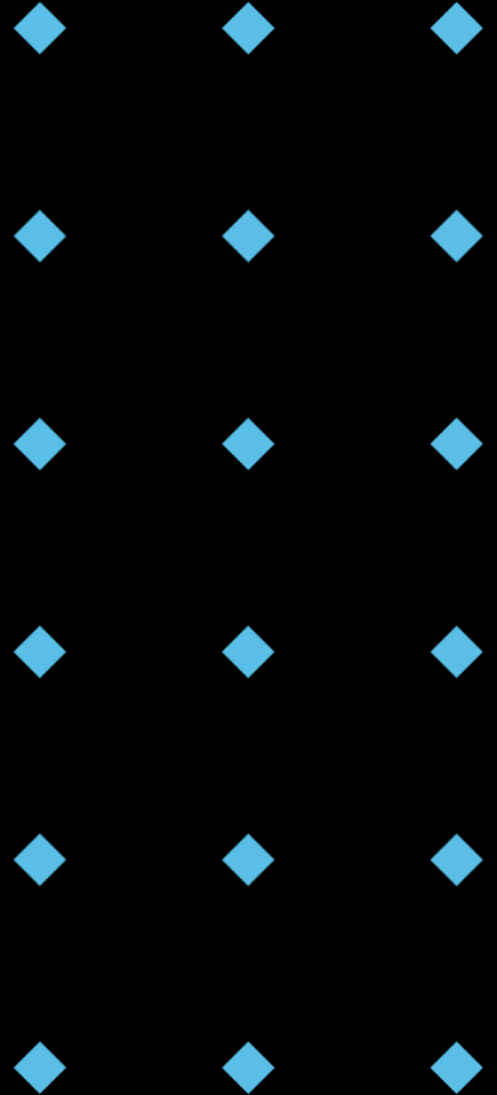


Discussion: AI-Driven Refactoring

Questions:

- ♦ What are the risks or downsides of using AI for refactoring? (Incorrect suggestions? Over-reliance? Losing understanding?)
- ♦ How much should we trust AI refactoring suggestions? What's the developer's role?

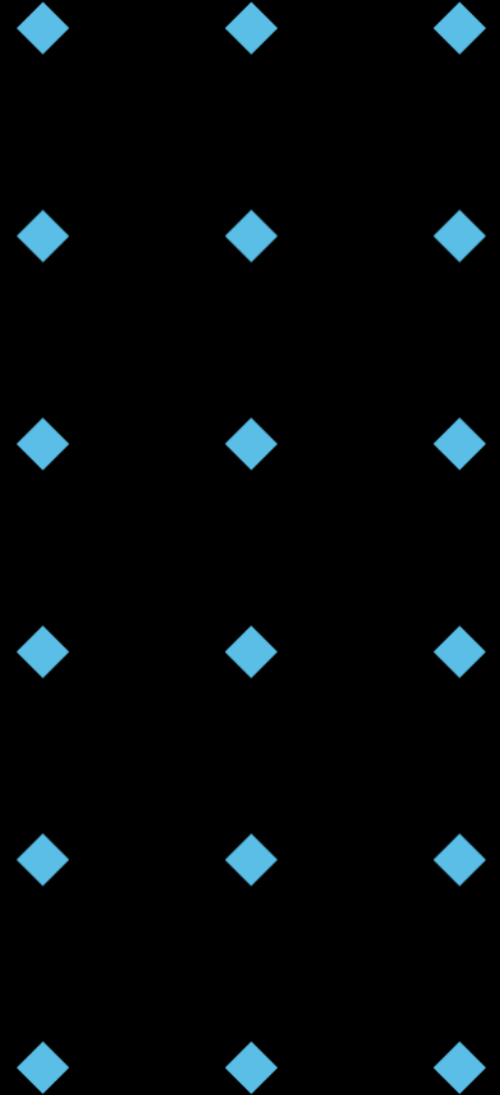
* Instructions: Take 2 minutes to think individually, then discuss with a partner for 3 minutes.



The Importance of Testing

- ◆ **Reliability:** Ensuring code works as expected.
- ◆ **Bug Prevention:** Catching errors early in development.
- ◆ **Maintainability:** Confidence to refactor/add features without breaking things.
- ◆ **Documentation:** Tests describe how code should behave.

AI's Role: Amplifying our ability to do both **effectively** and **efficiently**.



Unit Testing Fundamentals

- ◆ Definition: Testing **individual, isolated** components (units) of software. In OOP, a "unit" is often a method or a class.
- ◆ Goal: Verify that each part works correctly in **isolation**.
- ◆ Frameworks: Python's *unittest* module, *pytest*.
- ◆ Key Principles:
 - ◆ Tests should be independent.
 - ◆ Tests should be fast.
 - ◆ Tests should be repeatable.
 - ◆ Focus on public interfaces of classes/methods.
 - ◆ Test normal cases, boundary cases, and error cases.
- ◆ *unittest* Example: A testcase is created by subclassing *unittest.TestCase*. The three individual tests are defined with methods **whose names start with the letters *test***. This naming convention informs the test runner about which methods represent tests.

```
1 import unittest
2
3 class TestStringMethods(unittest.TestCase):
4
5     def test_upper(self):
6         self.assertEqual('foo'.upper(), 'FOO')
7
8     def test_isupper(self):
9         self.assertTrue('FOO'.isupper())
10        self.assertFalse('Foo'.isupper())
11
12    def test_split(self):
13        s = 'hello world'
14        self.assertEqual(s.split(), ['hello', 'world'])
15        # check that s.split fails when the separator is not a string
16        with self.assertRaises(TypeError):
17            s.split(2)
18
19 if __name__ == '__main__':
20     unittest.main()
```


How AI Helps Write Unit Tests

AI Copilot Capabilities:

- ◆ **Generating Test Boilerplate:** Can create the basic structure of a test class and test methods (e.g., using `unittest.TestCase` or `pytest` function conventions).
- ◆ **Creating Test Cases for Methods:** Given a method in your class, AI can often generate several test cases:
 - ◆ Testing the "happy path" (expected inputs and outputs).
 - ◆ Testing boundary conditions (e.g., empty lists, zero values).
 - ◆ Testing error conditions (e.g., invalid input raising exceptions).
- ◆ **Suggesting Assertions:** Can propose appropriate assertions (`assertEqual`, `assertTrue`, `assertRaises`) based on the code being tested.

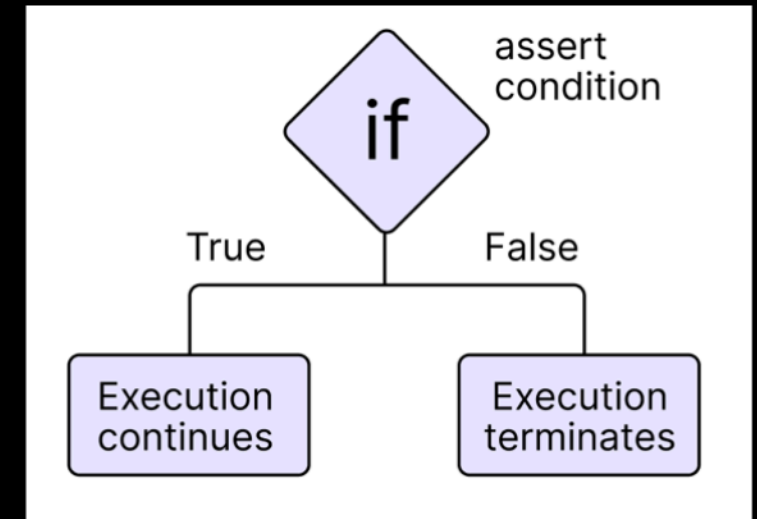


Image by Katalon Team, Jan 10, 2025,
<https://katalon.com/resources-center/blog/assert-in-python>

Example: Unit Test for a Simple Class

◆ Class to Test:

```
1 class InsufficientFundsError(Exception):
2     pass
3
4 class BankAccount:
5     def __init__(self, initial_balance=0):
6         if initial_balance < 0:
7             raise ValueError("Initial balance cannot be negative")
8         self._balance = initial_balance # Encapsulated attribute
9
10    def deposit(self, amount):
11        if amount <= 0:
12            raise ValueError("Deposit amount must be positive")
13        self._balance += amount
14
15    def withdraw(self, amount):
16        if amount <= 0:
17            raise ValueError("Withdrawal amount must be positive")
18        if amount > self._balance:
19            raise InsufficientFundsError("Insufficient funds")
20        self._balance -= amount
21
22    def get_balance(self):
23        return self._balance
```

[Provide Code]

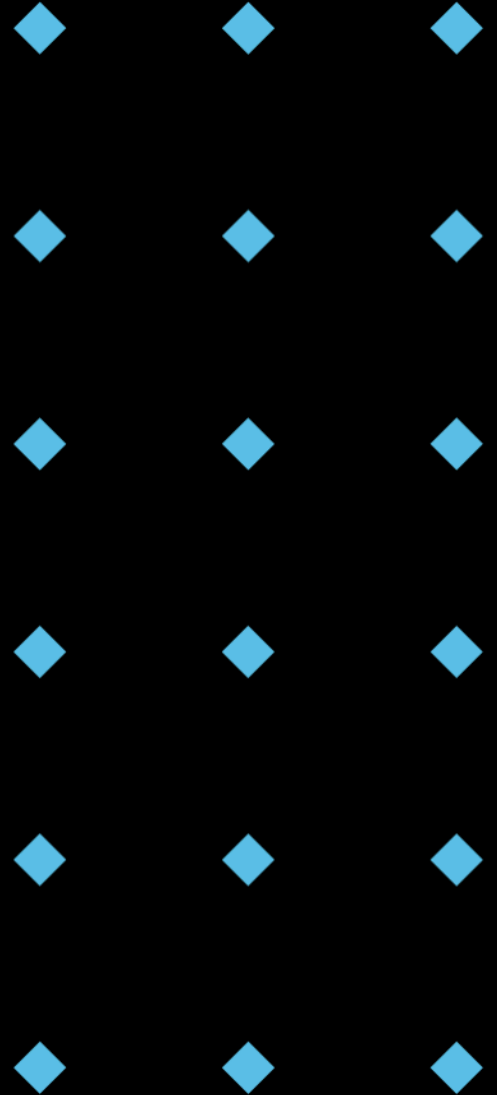
Generate unit tests for this class using *unittest*.

◆ Potential AI Output:

```
1 import unittest
2 # Assume BankAccount class and InsufficientFundsError are imported or defined above
3
4 class TestBankAccount(unittest.TestCase):
5
6     def test_initial_balance_default(self):
7         account = BankAccount()
8         self.assertEqual(account.get_balance(), 0)
9
10    def test_initial_balance_positive(self):
11        account = BankAccount(100)
12        self.assertEqual(account.get_balance(), 100)
13
14    def test_initial_balance_negative_raises_error(self):
15        with self.assertRaises(ValueError):
16            BankAccount(-50)
17
18    def test_deposit_positive_amount(self):
19        account = BankAccount(100)
20        account.deposit(50)
21        self.assertEqual(account.get_balance(), 150)
22
23    def test_deposit_zero_or_negative_raises_error(self):
24        account = BankAccount(100)
25        with self.assertRaises(ValueError):
26            account.deposit(0)
27        with self.assertRaises(ValueError):
28            account.deposit(-10)
29
30    def test_withdraw_sufficient_funds(self):
31        account = BankAccount(100)
32        account.withdraw(30)
33        self.assertEqual(account.get_balance(), 70)
```

Evaluating AI-Generated Tests

- ◆ Strengths:
 - ◆ **Speed:** Dramatically reduces time spent writing boilerplate.
 - ◆ **Coverage:** Often covers obvious paths and basic error conditions.
 - ◆ **Consistency:** Can help enforce a consistent testing style.
- ◆ Weaknesses / Need for Review:
 - ◆ **May Miss Subtleties:** Might not understand complex business logic or implicit requirements.
 - ◆ **Edge Case Blindness:** May not generate tests for obscure or non-obvious edge cases.
 - ◆ **Assertion Quality:** Might generate weak assertions (e.g., `assertIsNotNone` when a specific value is expected).
 - ◆ **Test Independence:** Might occasionally suggest tests that depend on each other (though usually good at isolation).
 - ◆ **Doesn't Understand Intent:** Tests what is written, not necessarily what was intended.
- ◆ Developer's Role: **Review, refine, and augment.** Add tests for **complex logic** and **critical edge cases** the AI missed. Ensure assertions are **meaningful**.



Test-Driven Development: TDD

- ◆ TDD: A development process where you write **tests before** writing the **actual code**.
- ◆ The Cycle (Red-Green-Refactor):
 - ◆ **Red**: Write a small, failing test for a new piece of functionality.
 - ◆ **Green**: Write the minimum amount of code necessary to make the test pass.
 - ◆ **Refactor**: Improve the code (and tests) while ensuring all tests still pass.
- ◆ Benefits: Ensures testability, provides rapid feedback, leads to simpler design, serves as documentation.

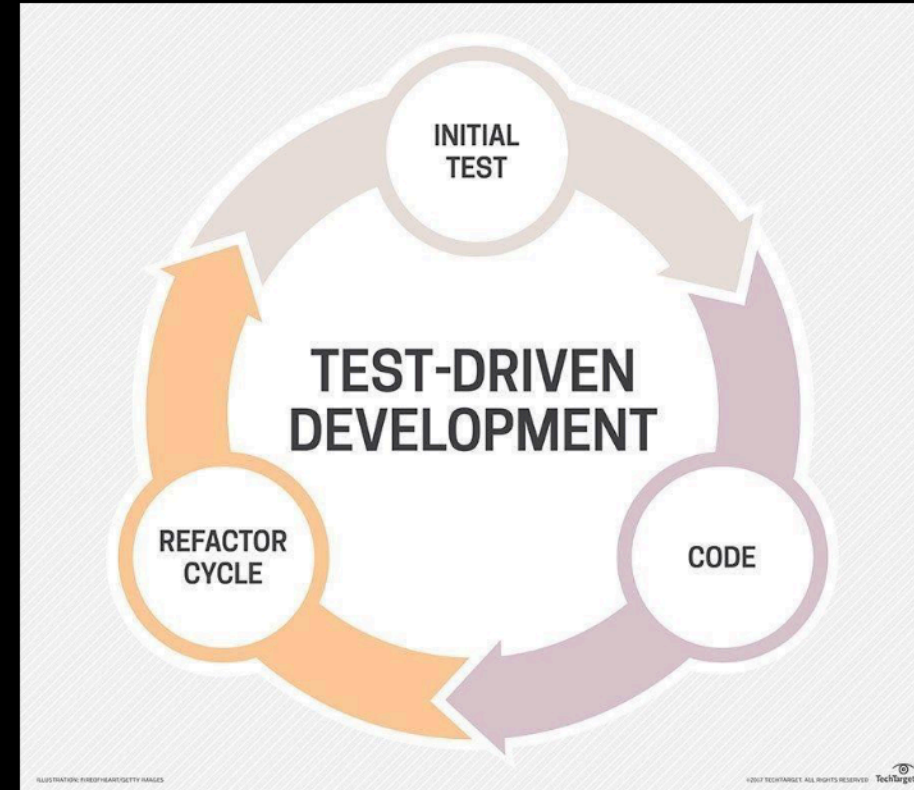
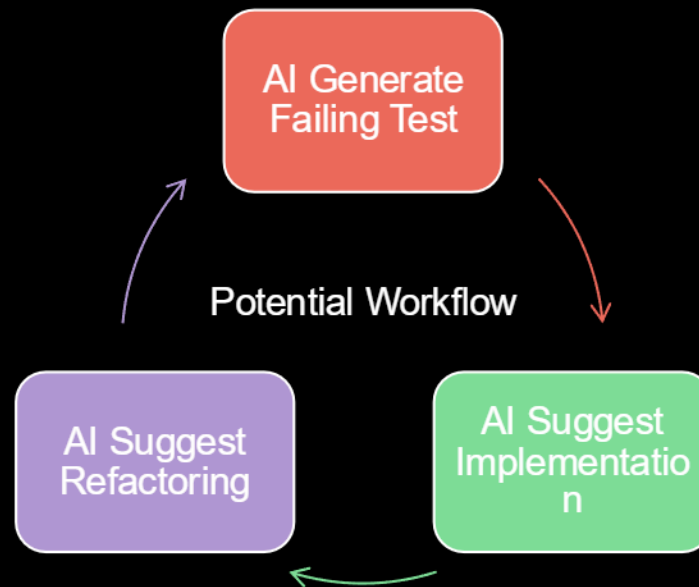


Image by Abhishek Srivastava, Mar 6, 2023,

<https://abhiappmobiledeveloper.medium.com/test-driven-development-tdd-42e43673eae9>

AI Supporting the TDD Cycle

- ◆ AI Copilot Capabilities:
 - ◆ **Red Phase:** Help formulate and write the failing test based on a description of the desired functionality. (e.g., "Write a `pytest` test for a function `calculate_discount(price, percentage)` that should return 90 for price=100, percentage=10").
 - ◆ **Green Phase:** Suggest minimal code implementation to pass the newly written test.
 - ◆ **Refactor Phase:** Suggest improvements or refactorings for the code that now passes the test (as discussed in Part 1).



TDD Example: Adding transfer Method to BankAccount

- ◆ Goal: Add a *transfer(self, other_account, amount)* method.

- ◆ Step 1 (RED - AI Assist):

[Provide Code]

Write a test for transferring funds successfully between two accounts.

- ◆ Potential AI Output:

```
1  def test_transfer_sufficient_funds(self):
2      acc1 = BankAccount(100)
3      acc2 = BankAccount(50)
4      acc1.transfer(acc2, 30) # This method doesn't exist yet!
5      self.assertEqual(acc1.get_balance(), 70)
6      self.assertEqual(acc2.get_balance(), 80)
```

- ◆ Run Test -> Fails (*AttributeError: 'BankAccount' object has no attribute 'transfer'*)

TDD Example: Adding transfer Method (Cont.)

- ◆ Step 2 (GREEN - AI Assist):

Implement the transfer method.

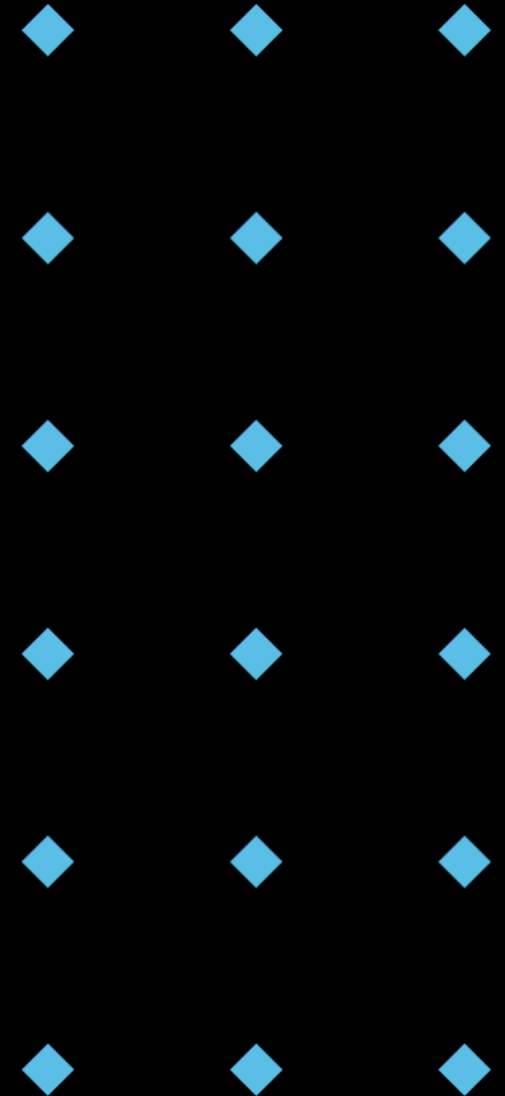
- ◆ Potential AI Output:

```
1 def transfer(self, other_account, amount):
2     # Minimal code to pass the specific test
3     if amount <= 0:
4         raise ValueError("Transfer amount must be positive") # Good guess by AI
5     if amount > self._balance:
6         raise InsufficientFundsError("Insufficient funds for transfer") # Good guess
7     # Simple implementation for now
8     self.withdraw(amount) # Re-use existing logic? Or maybe AI suggests direct modification:
9     # self._balance -= amount
10    other_account.deposit(amount) # Re-use? Or maybe:
11    # other_account._balance += amount # Direct access might be suggested initially
```

- ◆ Run Tests -> `test_transfer_sufficient_funds` now Passes! (GREEN state)

TDD Example: Adding transfer Method (Cont.)

- ◆ Step 3 (Refactor - AI Assist):
 - ◆ Look at the new transfer method. Is it clean?
 - ◆ Maybe the AI initially suggested direct balance manipulation (`_balance = _balance - amount`).
 - ◆ Prompt: "Refactor transfer to use deposit/withdraw methods."
 - ◆ AI Suggests changing the implementation to call `self.withdraw(amount)` and `other_account.deposit(amount)`.
 - ◆ Run Tests -> Still Passing! (Refactoring complete for now)
- ◆ Step 4 (Repeat Cycle):
 - ◆ Write a test for transferring insufficient funds (RED).
 - ◆ AI might help implement the check (or it might already be covered if withdraw was used) (GREEN).
 - ◆ Refactor if needed.
 - ◆ Write a test for transferring a negative amount (RED), etc.

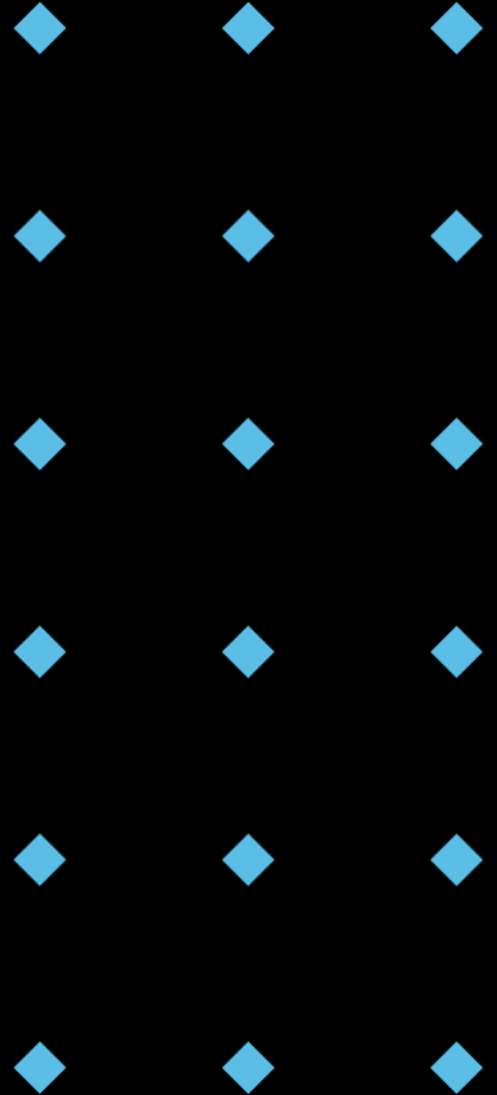


Discussion: TDD with AI

Questions:

- ◆ Do you think using AI makes adopting TDD easier or harder? Why?
- ◆ What are the risks of relying too heavily on AI for generating both the tests and the code in a TDD cycle?
- ◆ How does TDD change the way you think about designing your classes and methods?

* Instructions: Take 2 minutes to think individually, then discuss with a partner for 3 minutes.



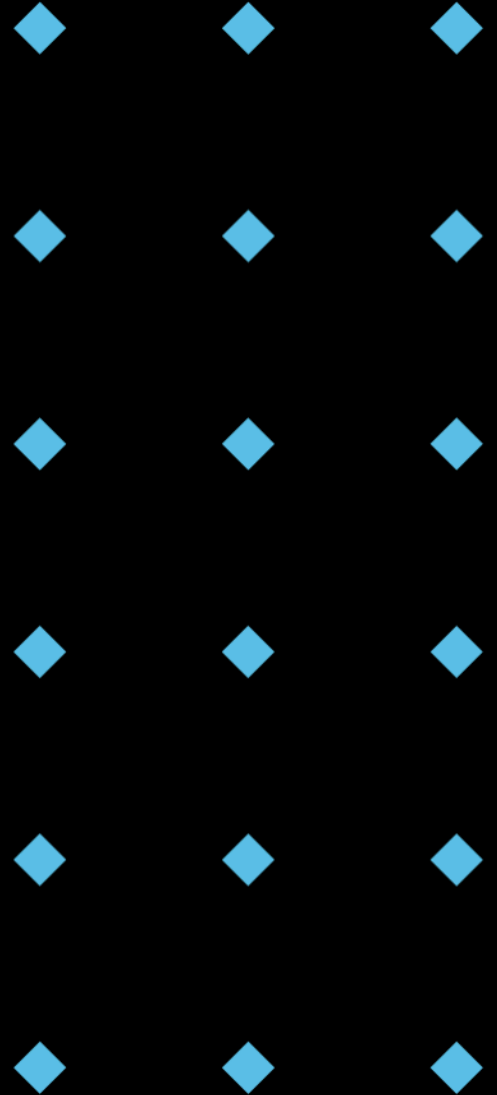
The Dependency Problem in Unit Testing

- ◆ Scenario: A class *OrderProcessor* depends on a *PaymentGateway* service and a *DatabaseConnector*.
- ◆ Testing Challenges:
 - ◆ We don't want to make real payments in tests!
 - ◆ We don't want to connect to a real database for unit tests (slow, requires setup, state issues).
 - ◆ We only want to test the logic of *OrderProcessor*, not the gateway or DB.

```
1 class OrderProcessor:
2     def __init__(self, payment_gateway, db_connector):
3         self.gateway = payment_gateway
4         self.db = db_connector
5
6     def process_order(self, order_details):
7         # 1. Charge the customer via payment gateway
8         payment_ok = self.gateway.charge(order_details['customer_id'], order_details['amount'])
9         if payment_ok:
10            # 2. Save the order to the database
11            self.db.save_order(order_details)
12            return "Order processed successfully"
13        else:
14            return "Payment failed"
```

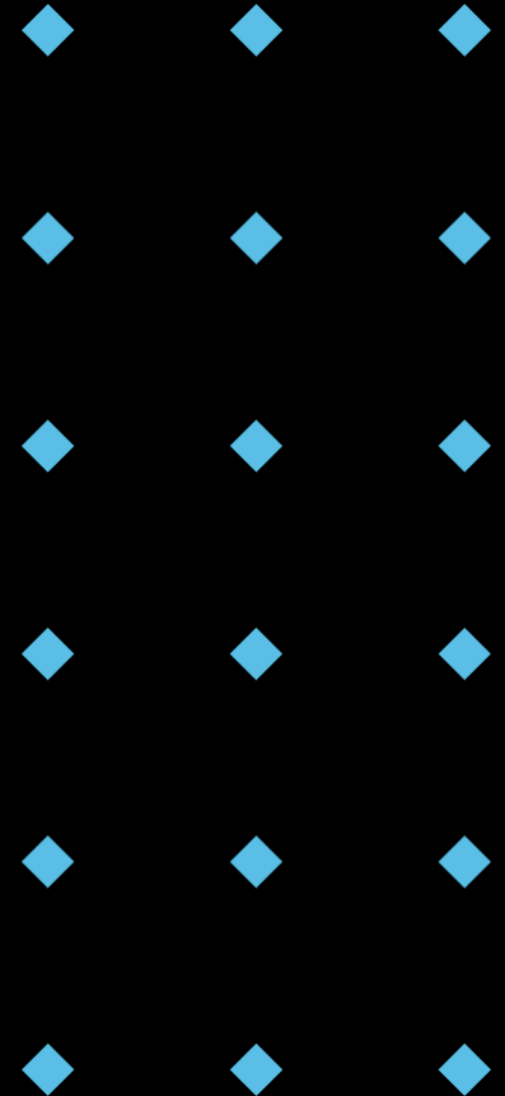
Mock Objects to the Rescue

- ◆ Concept: **Mock objects** are 'fake' objects that **simulate** the behaviour of real dependencies.
- ◆ Python's *unittest.mock*: The standard library module for creating mocks.
 - ◆ *Mock*: A flexible mock object.
 - ◆ *MagicMock*: A Mock subclass that pre-configures magic methods (like `__init__`).
 - ◆ *patch*: A decorator or context manager to temporarily replace objects during a test.
- ◆ How it Works:
 - ◆ Create a mock object that looks like the real dependency (has the same methods).
 - ◆ Configure the mock's methods to return specific values. E.g., `mock_gateway.charge.return_value = True`.
 - ◆ Configure mocks to check if they were called correctly. E.g., `mock_db.save_order.assert_called_once_with(order_details)`.
 - ◆ Inject the mock into the class being tested instead of the real dependency.



AI Assistance in Creating Mocks

- ◆ Boilerplate Generation:
 - ◆ Generating *unittest.mock* import statements.
 - ◆ Creating *Mock* or *MagicMock* instances.
 - ◆ Setting up *@patch* decorators or with *patch(...)* context managers.
- ◆ Configuration Suggestion:
 - ◆ Suggesting *return_value* settings based on how the dependency is used.
 - ◆ Suggesting *side_effect* settings (e.g., to simulate exceptions).
 - ◆ Generating assertion calls like *assert_called_once_with(...)*.
- ◆ Prompting:
 - ◆ "Mock the *PaymentGateway* dependency"
 - ◆ "Create a mock for *DatabaseConnector* using *unittest.mock*"
 - ◆ "Use patch to mock '*mymodule.PaymentGateway*' in this test"



Example: AI Mocking *PaymentGateway*

- Test Goal: Test *OrderProcessor.process_order* when payment succeeds.

```
1 class OrderProcessor:
2     def __init__(self, payment_gateway, db_connector):
3         self.gateway = payment_gateway
4         self.db = db_connector
5
6     def process_order(self, order_details):
7         # 1. Charge the customer via payment gateway
8         payment_ok = self.gateway.charge(order_details['customer_id'],
9                                         order_details['amount'])
10        if payment_ok:
11            # 2. Save the order to the database
12            self.db.save_order(order_details)
13            return "Order processed successfully"
14        else:
15            return "Payment failed"
```

[Provide Code]

Write a *unittest* test for *process_order* success, mocking dependencies.

- Potential AI Output:

```
1 import unittest
2 from unittest.mock import MagicMock # AI adds import
3 # Assume OrderProcessor, PaymentGateway, DatabaseConnector are defined/
4 imported
5
6 class TestOrderProcessor(unittest.TestCase):
7
8     def test_process_order_success(self):
9         # 1. Create mock objects (AI Generated)
10        mock_gateway = MagicMock()
11        mock_db = MagicMock()
12
13        # 2. Configure mock behavior (AI Suggested/Generated)
14        mock_gateway.charge.return_value = True # Simulate successful charge
15
16        # 3. Create instance with mocks (AI uses mocks)
17        order_processor = OrderProcessor(mock_gateway, mock_db)
18        order_details = {'customer_id': 123, 'amount': 100, 'items': ['apple',
19                                                                    'banana']}
20
21        # 4. Call the method under test
22        result = order_processor.process_order(order_details)
23
24        # 5. Assert results and mock interactions (AI Generated)
25        self.assertEqual(result, "Order processed successfully")
26        mock_gateway.charge.assert_called_once_with(123, 100)
27        mock_db.save_order.assert_called_once_with(order_details)
```


Example: AI Mocking Payment Failure

- Test Goal: Test `OrderProcessor.process_order` when payment fails.

```
1 class OrderProcessor:
2     def __init__(self, payment_gateway, db_connector):
3         self.gateway = payment_gateway
4         self.db = db_connector
5
6     def process_order(self, order_details):
7         # 1. Charge the customer via payment gateway
8         payment_ok = self.gateway.charge(order_details['customer_id'],
9                                         order_details['amount'])
10        if payment_ok:
11            # 2. Save the order to the database
12            self.db.save_order(order_details)
13            return "Order processed successfully"
14        else:
15            return "Payment failed"
```

[Provide Code]

Test `process_order` when payment gateway charge fails.

- Potential AI Output:

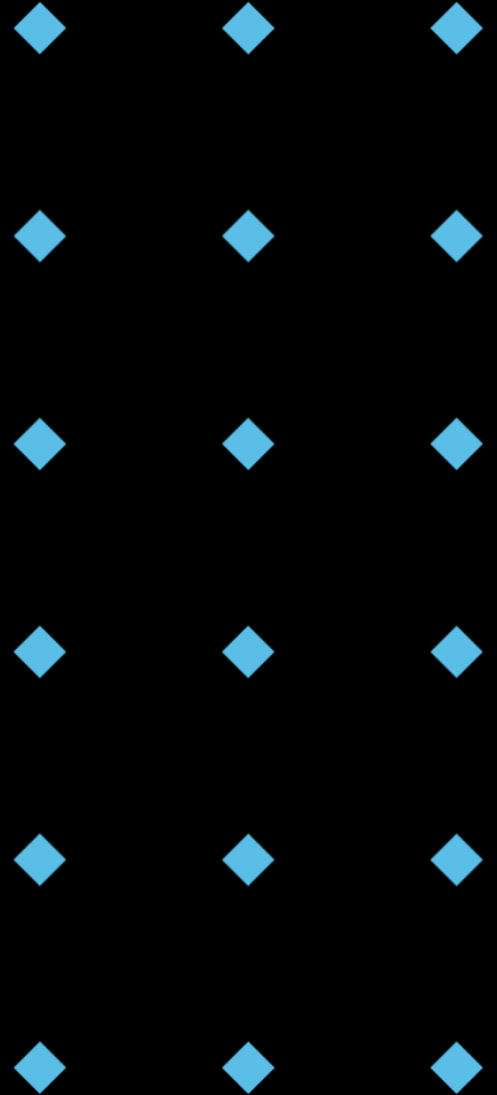
```
27 # (Inside TestOrderProcessor class)
28 def test_process_order_payment_failure(self):
29     # 1. Create mocks
30     mock_gateway = MagicMock()
31     mock_db = MagicMock() # Still need db mock, even if not used
32
33     # 2. Configure mock behavior for failure
34     mock_gateway.charge.return_value = False # Simulate failed charge
35
36     # 3. Create instance with mocks
37     order_processor = OrderProcessor(mock_gateway, mock_db)
38     order_details = {'customer_id': 456, 'amount': 200}
39
40     # 4. Call method
41     result = order_processor.process_order(order_details)
42
43     # 5. Assert results and mock interactions
44     self.assertEqual(result, "Payment failed")
45     mock_gateway.charge.assert_called_once_with(456, 200)
46     mock_db.save_order.assert_not_called() # Important check!
```


Discussion: AI and Mocking

Questions:

- ◆ What are the risks of letting AI generate mock configurations and assertions? (e.g., mocking too much, incorrect assertions).
- ◆ Does using AI make mocking less daunting?
- ◆ Where might you still need significant manual effort when mocking complex dependencies? (e.g., simulating complex state changes in a mock).

* Instructions: Take 2 minutes to think individually, then discuss with a partner for 3 minutes.



Finding the Tricky Cases: Edge Cases

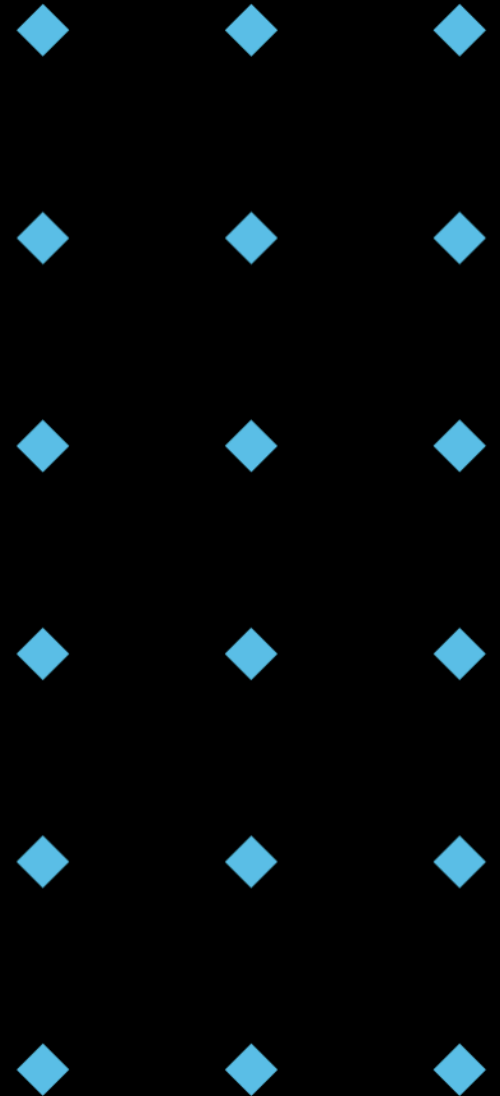
- ◆ **Edge Cases:** Inputs or scenarios that are unusual, extreme, or represent boundary conditions.
 - ◆ Empty inputs (empty strings, lists, dictionaries)
 - ◆ Null or None values
 - ◆ Zero values
 - ◆ Very large or very small numbers
 - ◆ Data exactly at a boundary (e.g., minimum/maximum allowed values)
 - ◆ Unexpected data types
 - ◆ Concurrency issues (less common for AI suggestion)
- ◆ Importance: Bugs often hide in **edge cases** because they are less common and easily overlooked during development and basic testing.



Image by pr3579, Reddit, Inc, Aug 4, 2021,
https://www.reddit.com/r/ProgrammerHumor/comments/oxq5no/edge_cases_be_like/

AI Assistance in Finding Edge Cases

- ◆ **Code Analysis:** AI can analyse code paths, conditions (if, while), and data types.
- ◆ **Pattern Recognition:** Based on its training data, AI might recognize patterns that commonly lead to edge case bugs.
- ◆ **Input Suggestion:**
 - ◆ Suggesting boundary values based on comparisons (e.g., if $x < 10$, suggest testing $x=9$, $x=10$).
 - ◆ Suggesting "empty" or "null" values (e.g., `[]`, `""`, `None`).
 - ◆ Suggesting different data types to test robustness.
- ◆ **Prompting:**
 - ◆ "What are potential edge cases for this function?"
 - ◆ "Suggest edge case inputs for the '`amount`' parameter."
 - ◆ "Are there any boundary conditions I should test here?"



Example: Finding Edge Cases

- Context: Our Library class has a `borrow_book(member_id, isbn)` method.

```
47 def borrow_book(self, member_id: int, isbn: str):
48     """Allow a member to borrow a book if available."""
49     if member_id not in self.members:
50         print(f"Error: Member with ID {member_id} not found.")
51         return
52     if isbn not in self.books:
53         print(f"Error: Book with ISBN {isbn} not found.")
54         return
55
56     member = self.members[member_id]
57     book = self.books[isbn]
58
59     if book.is_borrowed:
60         print(f"Sorry, '{book.title}' is already borrowed.")
61     else:
62         book.is_borrowed = True
63         member.borrowed_books.append(book)
64         print(f"{member.name} borrowed '{book.title}'.")
```

[Provide Code]

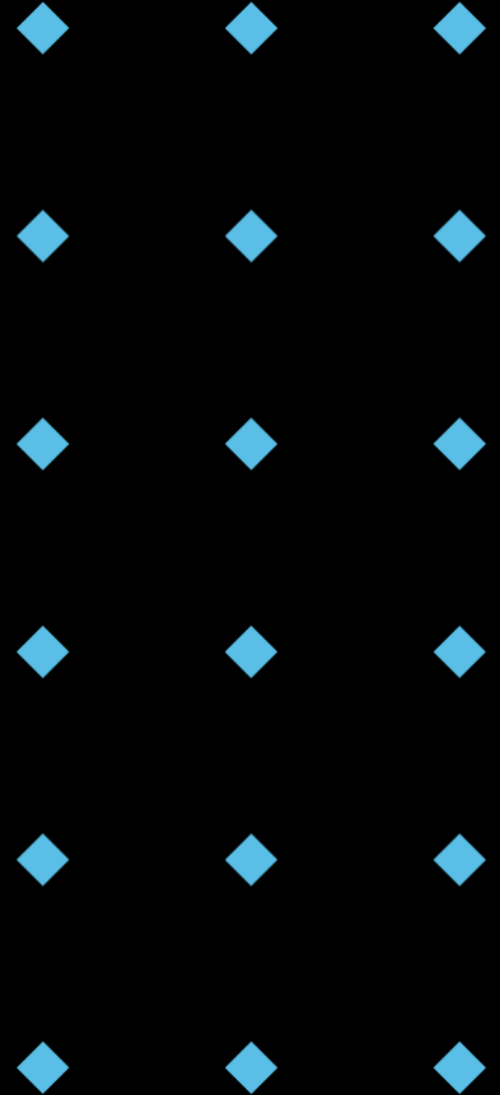
What are potential edge cases and error conditions to test for the `Library's borrow_book(member_id, isbn)` method, considering it needs to find the member, find the book, check availability, and update states?

- Potential AI Output:

- *Member ID does not exist.*
- *ISBN does not exist.*
- *Book exists but is already borrowed.*
- *Member already has the maximum allowed number of books (if such a rule exists).*
- *Member ID or ISBN provided is None or invalid type.*
- *Attempting to borrow the same book twice by the same member.*
- *What if the `member.borrowed_books` list or `library.books` list is unexpectedly empty?*

Limitations of AI Edge Case Finding

- ◆ **Context Blindness:** AI doesn't fully understand the business domain or implicit requirements.
- ◆ **Truly Novel Cases:** AI relies on patterns learned from data; it may struggle to find truly unique or domain-specific edge cases it hasn't seen before.
- ◆ **Complex Interactions:** May miss edge cases arising from interactions between multiple components or systems.
- ◆ **Requires Human Insight:** Still requires developer experience, domain knowledge, and critical thinking to anticipate the most relevant and impactful edge cases. AI is a helpful tool, not a replacement for thought.



Understanding Test Coverage

- ◆ Definition: A metric used to measure the **degree** to which the **source code** of a program is **executed** when a particular test suite is run.
- ◆ Common Types:
 - ◆ **Statement Coverage:** % of statements executed.
 - ◆ **Branch Coverage:** % of branches (e.g., if/else outcomes) executed. (Often more valuable than statement).
 - ◆ **Function/Method Coverage:** % of functions/methods called.
- ◆ Tools: [coverage.py](#) is the standard Python tool.
 - ◆ Run tests under [coverage run -m unittest discover](#).
 - ◆ Generate reports: [coverage report](#) (console), [coverage html](#) (HTML report).
- ◆ Goal: Identify parts of the code **not exercised** by tests. High coverage is necessary, but **not sufficient**, for good testing.

Coverage report: 96%

Files

Functions

Classes

coverage.py v7.8.0, created at 2025-04-08 22:16 +1000

File ▲	statements	missing	excluded	coverage
bank.py	19	1	0	95%
utest.py	38	1	0	97%
Total	57	2	0	96%

coverage.py v7.8.0, created at 2025-04-08 22:16 +1000

AI Assistance in Improving Coverage

- ◆ Identifying Untested Code: While `coverage.py` shows what is untested, AI might help understand why.
- ◆ Suggesting Tests for Gaps:
 - ◆ **Analyse** uncovered lines or branches.
 - ◆ **Suggest** specific inputs or scenarios that would execute that code.
 - ◆ **Generate** test method skeletons targeting those gaps.
- ◆ Prompting:
 - ◆ Select an uncovered block of code: "Write a test that covers this code block."
 - ◆ Analyse coverage report findings: "Suggest tests to improve coverage for the '`process_order`' method's error handling."

```
Coverage for bank.py: 95%
19 statements  18 run  1 missing  0 excluded
« prev  ^ index  » next  coverage.py v7.8.0, created at 2025-04-08 22:16 +1000

1 class InsufficientFundsError(Exception):
2     pass
3
4 class BankAccount:
5     def __init__(self, initial_balance=0):
6         if initial_balance < 0:
7             raise ValueError("Initial balance cannot be negative")
8         self._balance = initial_balance # Encapsulated attribute
9
10    def deposit(self, amount):
11        if amount <= 0:
12            raise ValueError("Deposit amount must be positive")
13        self._balance += amount
14
15    def withdraw(self, amount):
16        if amount <= 0:
17            raise ValueError("Withdrawal amount must be positive")
18        if amount > self._balance:
19            raise InsufficientFundsError("Insufficient funds")
20        self._balance -= amount
21
22    def get_balance(self):
23        return self._balance

« prev  ^ index  » next  coverage.py v7.8.0, created at 2025-04-08 22:16 +1000
```


Example: AI Closing a Coverage Gap

- Scenario: *coverage.py* report shows the *InsufficientFundsError* block in *BankAccount.withdraw* is not covered.

```
49 # ... inside BankAccount ...
50 def withdraw(self, amount):
51     # ... checks for positive amount ...
52     if amount > self._balance: # This condition was never True in tests
53         raise InsufficientFundsError("Insufficient funds") # <--- RED
54                                     LINE IN COVERAGE REPORT
55     self._balance -= amount
```

[Provide Code]
Write a test to cover the *raise InsufficientFundsError* line.

AI Generation:

```
57 # (Inside TestBankAccount class)
58 def test_withdraw_insufficient_funds_covers_exception(self): # AI
59     # generates new test
60     account = BankAccount(50) # Start with balance less than withdrawal
61     with self.assertRaises(InsufficientFundsError):
62         account.withdraw(100) # Attempt to withdraw more than balance
63     # Optional: Check balance remains unchanged
64     self.assertEqual(account.get_balance(), 50)
```

Coverage report: 98%

Files Functions Classes

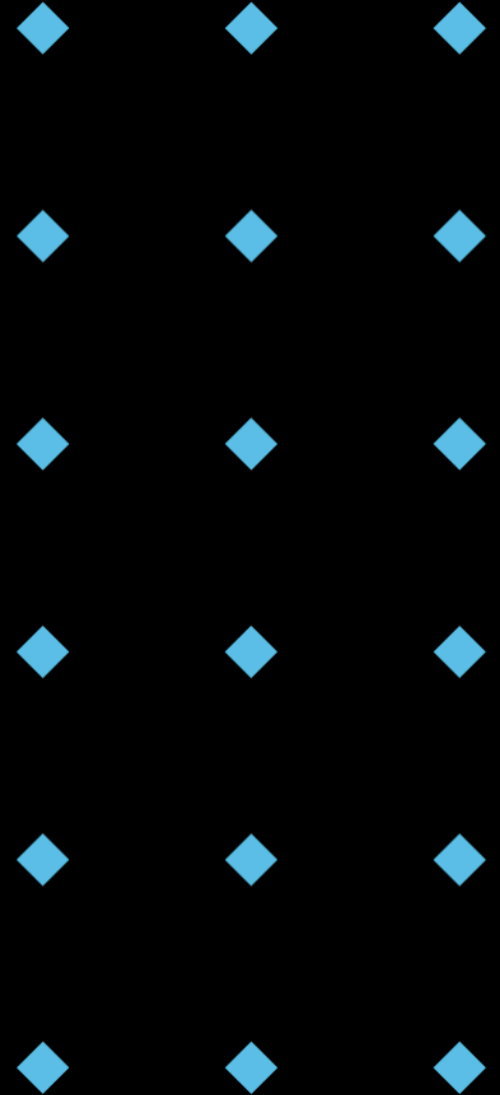
coverage.py v7.8.0, created at 2025-04-08 22:22 +1000

File ▲	statements	missing	excluded	coverage
bank.py	19	0	0	100%
utest.py	42	1	0	98%
Total	61	1	0	98%

coverage.py v7.8.0, created at 2025-04-08 22:22 +1000

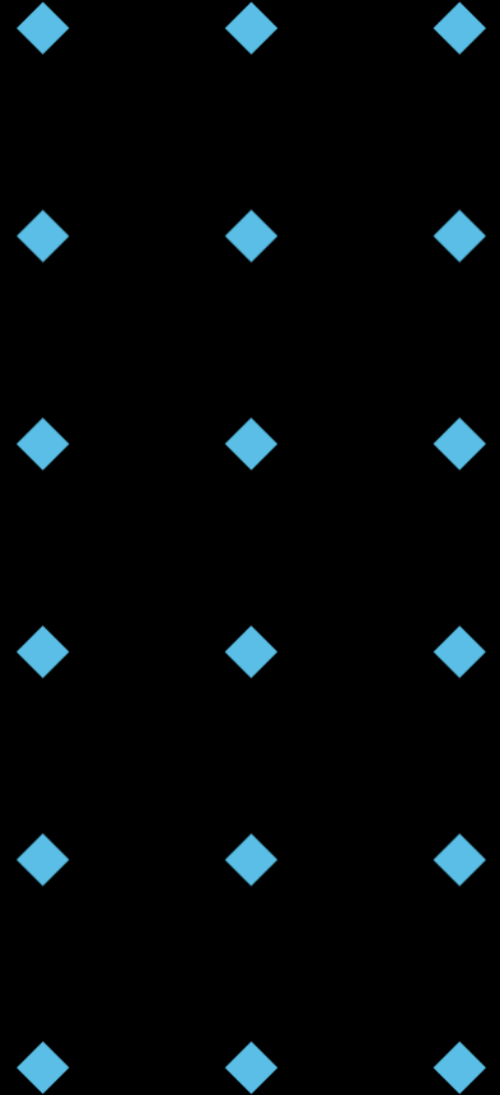
Coverage Goals: Use AI Wisely

- ◆ **Don't Chase 100% Blindly:** Focus on covering critical logic, complex branches, and error handling. Testing trivial getters/setters might not add much value.
- ◆ **AI Helps Reach Goals Faster:** Use AI to quickly generate tests for obvious gaps identified by coverage tools.
- ◆ **Human Judgment Still Key:** Evaluate if covering a specific line adds real value. Is the uncovered code important? Is it complex? Could it realistically fail?
- ◆ **Focus on Meaningful Tests:** A test suggested by AI just to cover a line might not be a good test if it doesn't check meaningful behaviour or assertions. Review and refine.



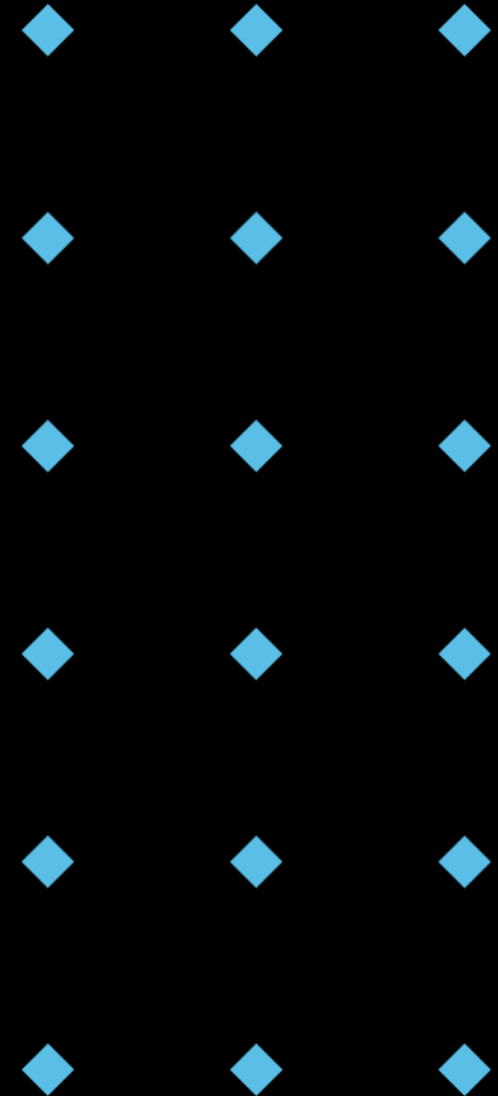
Integration Testing Overview

- ◆ Goal: Verify that different modules or services (classes, components, microservices) work **together** as expected.
- ◆ Focus: **Interactions, interfaces, data flow** between **units**.
- ◆ Challenges:
 - ◆ Requires setting up **multiple components** (e.g., database, web service, multiple classes).
 - ◆ Tests are typically **slower** than unit tests.
 - ◆ Failures can be **harder** to pinpoint (was it Component A, Component B, or the interaction?).
 - ◆ Often involves **real dependencies** (or near-real substitutes like test databases).



AI Assistance in Integration Testing

- ◆ Possible Assistance Areas:
 - ◆ **Test Scenario Generation:** Suggesting high-level scenarios based on component interactions (e.g., "Test order placement followed by inventory check").
 - ◆ **Configuration/Setup Help:** Potentially assisting with generating configuration files (e.g., Docker Compose, database setup scripts) - though often complex.
 - ◆ **Log Analysis:** AI might help analyse logs from failed integration tests to pinpoint issues (emerging area).
 - ◆ **API Interaction Tests:** Generating basic client code to interact with APIs based on specifications.
- ◆ Significant Limitations:
 - ◆ Requires **deep understanding** of system architecture and deployment.
 - ◆ Environment setup is highly **context-specific**.
 - ◆ Debugging interaction failures often needs **human insight**.
 - ◆ **Less mature** application of AI compared to unit testing/refactoring.



Discussion: Testing Pyramid

The Testing Pyramid:

- ◆ Top: Very few, very slow End-to-End / UI Tests.
- ◆ Middle: Fewer, slower Integration Tests.
- ◆ Base: Lots of fast, simple Unit Tests.

Questions:

- ◆ Why is this pyramid shape generally recommended? (Cost, speed, feedback loop).
- ◆ Where does AI assistance seem most impactful based on today's session (Unit, Integration)? Why?
- ◆ How do you decide when a unit test is sufficient versus needing an integration test?

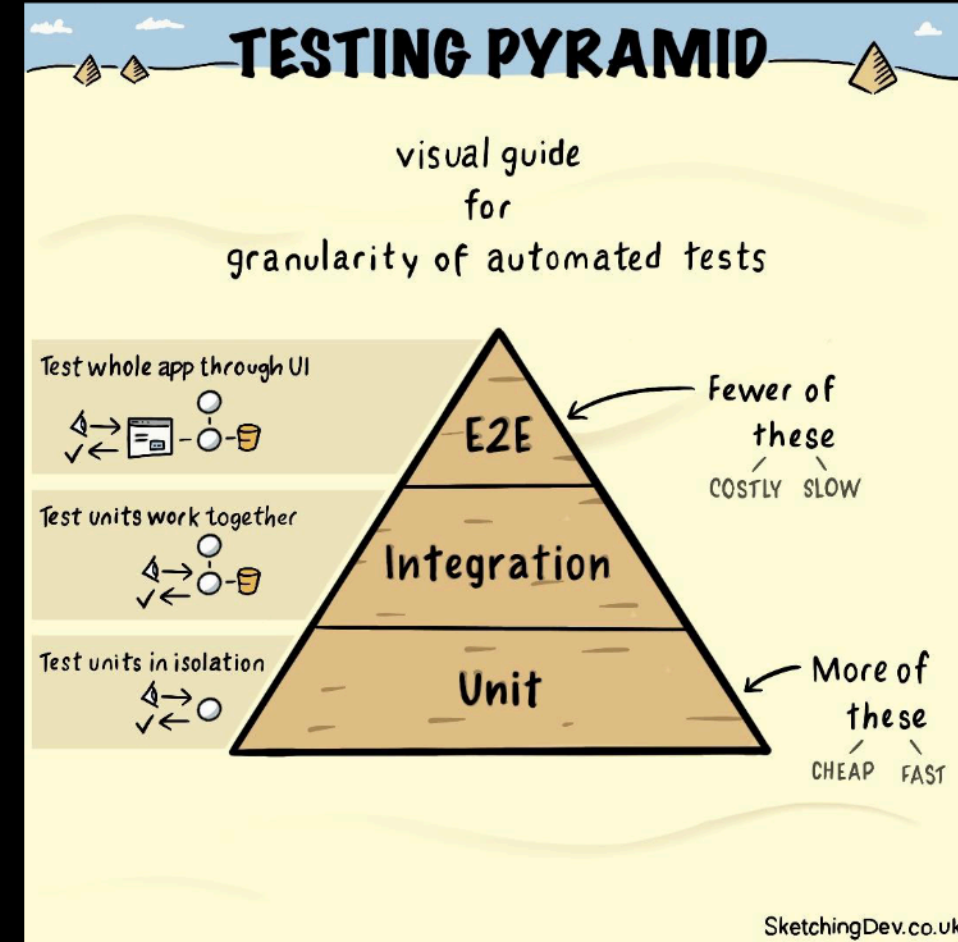


Image from SketchingDev, 2023,
<https://sketchingdev.co.uk/sketchnotes/testing-pyramid.html>

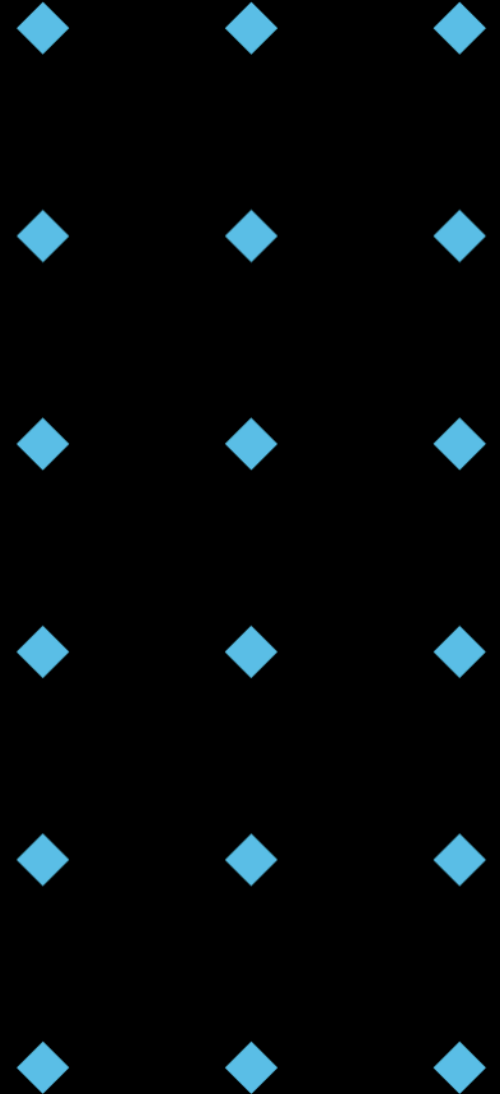
AI is a Tool, YOU are the Developer

AI Does:

- ◆ Generate suggestions.
- ◆ Automate boilerplate.
- ◆ Identify common patterns.
- ◆ Accelerate tasks.

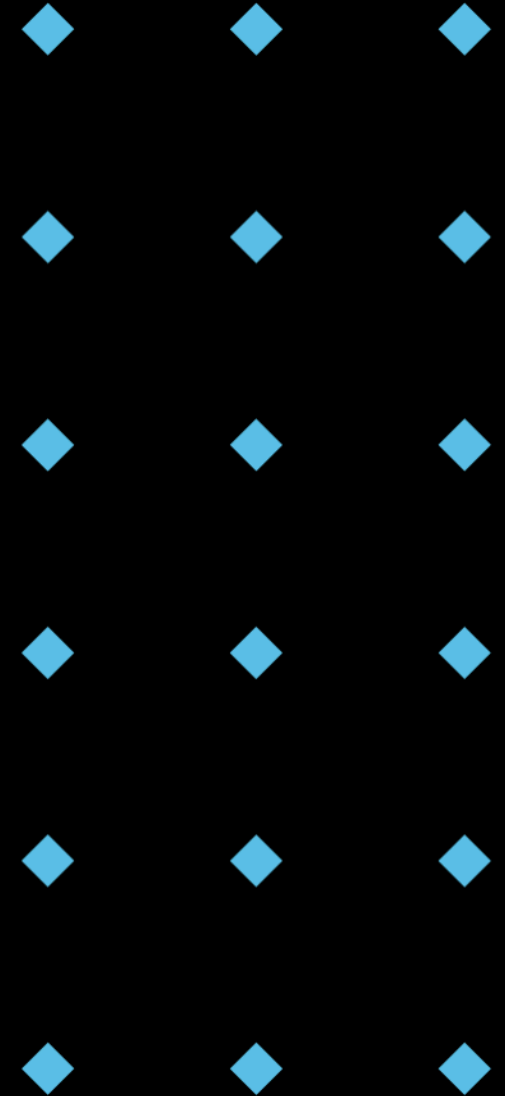
You (The Developer) Must:

- ◆ **Understand:** The problem, the code, the principles (OOP, testing, optimization).
- ◆ **Critically Evaluate:** AI suggestions (correctness, relevance, trade-offs).
- ◆ **Design:** The overall architecture and test strategy.
- ◆ **Debug:** Complex issues AI might miss or introduce.
- ◆ **Own:** The quality and performance of the final product.
- ◆ **Don't blindly trust;** use AI to enhance your own expertise.

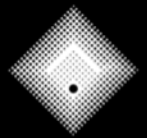


Key Takeaways

- ◆ Refactoring: AI suggests improvements (Extract Method, Simplify Logic), but verify suggestions and prioritize clarity/maintainability. Measure performance changes.
- ◆ Unit Testing: AI generates test boilerplate and basic cases quickly. Review, refine, and add complex/edge case tests manually.
- ◆ TDD: AI can assist in all phases (Red, Green, Refactor), potentially speeding up the cycle but requiring developer guidance.
- ◆ Mocking: AI excels at generating mock setup code (`unittest.mock`, `patch`), reducing verbosity. Ensure mock behaviour and assertions are correct.
- ◆ Edge Cases: AI can suggest potential edge cases by analysing code, but human insight and domain knowledge remain crucial.
- ◆ Coverage: AI helps generate tests to cover gaps identified by tools like `coverage.py`. Focus on meaningful coverage, not just the percentage.
- ◆ Integration Testing: AI's role is currently more limited but may evolve, potentially helping with scenario generation or log analysis.



Q & A



**VICTORIA
UNIVERSITY**

MELBOURNE AUSTRALIA



Lab

